# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA17**

---

**COURSE OUTCOME COVERED**

***CO4:*** *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

*Assessment Tool Weightage: 40%*

---

**Duration : 45 Minutes**

**QUESTIONS: 2** **Total Marks:20**

Annexure A

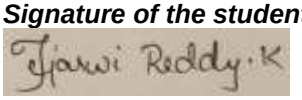
**INTEGRATED EXPERIMENTS**

---

***Code of Conduct:***

*I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***



Annexure A

## INTEGRATED EXPERIMENTS

---

**Experiment 1:** Decision Tree Classification

***Objective:*** Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

**(a)** Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

# Add target column
df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Separate features and target
X = df.drop("target", axis=1)
y = df["target"]

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Activate Windows
Go to Settings to activate Windows.

```
    stratify=y
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
First 5 rows:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target                 int64
dtype: object
```

Activate Windows

```
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target                 int64
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64

Training samples: 120
Testing samples: 30
```

Activate Windows

**(b)** Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

```
# Create Decision Tree classifier using Gini Index
dt_model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train the model
dt_model.fit(X_train, y_train)

# Print tree structure
print("Decision Tree Structure:")
from sklearn.tree import export_text

tree_rules = export_text(
    dt_model,
    feature_names=list(X.columns)
)

print(tree_rules)

# Identify root node
root_index = dt_model.tree_.feature[0]
```

Activate Windows

```
root_index = dt_model.tree_.feature[0]
root_feature = X.columns[root_index]

print("\nRoot Node:", root_feature)
```

```
Decision Tree Structure:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- sepal length (cm) <= 6.15
|   |   |   |   |--- sepal width (cm) <= 2.45
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- sepal width (cm) >  2.45
|   |   |   |   |   |--- class: 1
|   |   |   |--- sepal length (cm) >  6.15
|   |   |   |   |--- class: 2
|   |--- petal width (cm) >  1.65
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- sepal width (cm) <= 3.00
|   |   |   |   |--- class: 2
|   |   |   |--- sepal width (cm) >  3.00
|   |   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: 2


Root Node: petal length (cm)
```

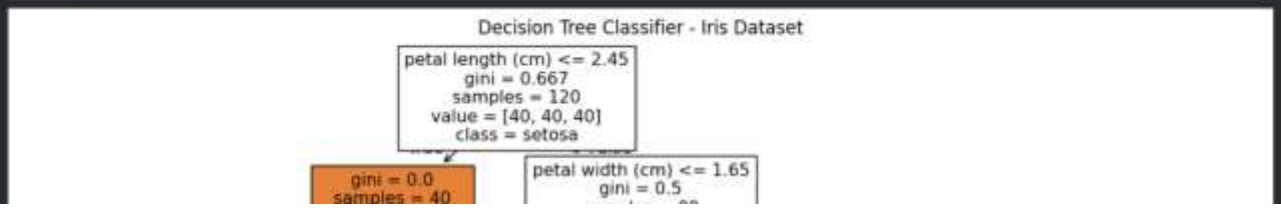
Activate Windows

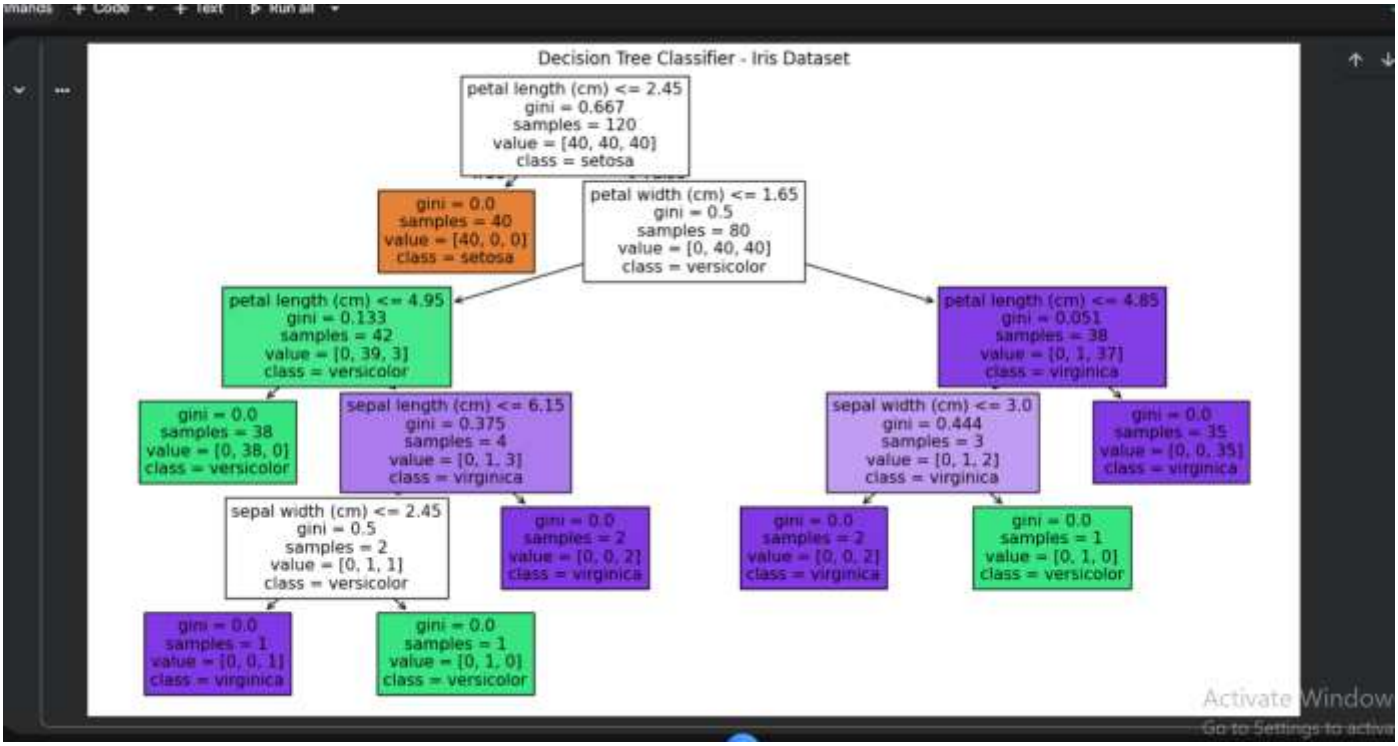
```
Root Node: petal length (cm)
```

```
plt.figure(figsize=(15, 8))

plot_tree(
    dt_model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree Classifier - Iris Dataset")
plt.show()
```






**(c)** Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Model Evaluation

```
# Predict test data
y_pred = dt_model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# Find misclassified instances
misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]

# Convert class numbers to class names
misclassified["Actual"] = misclassified["Actual"].map(
```

Activate Windows
Go to Settings to activate Windows.

[7] ✓ 0s

```
misclassified["Actual"] = misclassified["Actual"].map(
    dict(enumerate(iris.target_names))
)

misclassified["Predicted"] = misclassified["Predicted"].map(
    dict(enumerate(iris.target_names))
)

print("\nMisclassified Instances:")
print(misclassified)
```

```
Accuracy: 0.9333333333333333

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30

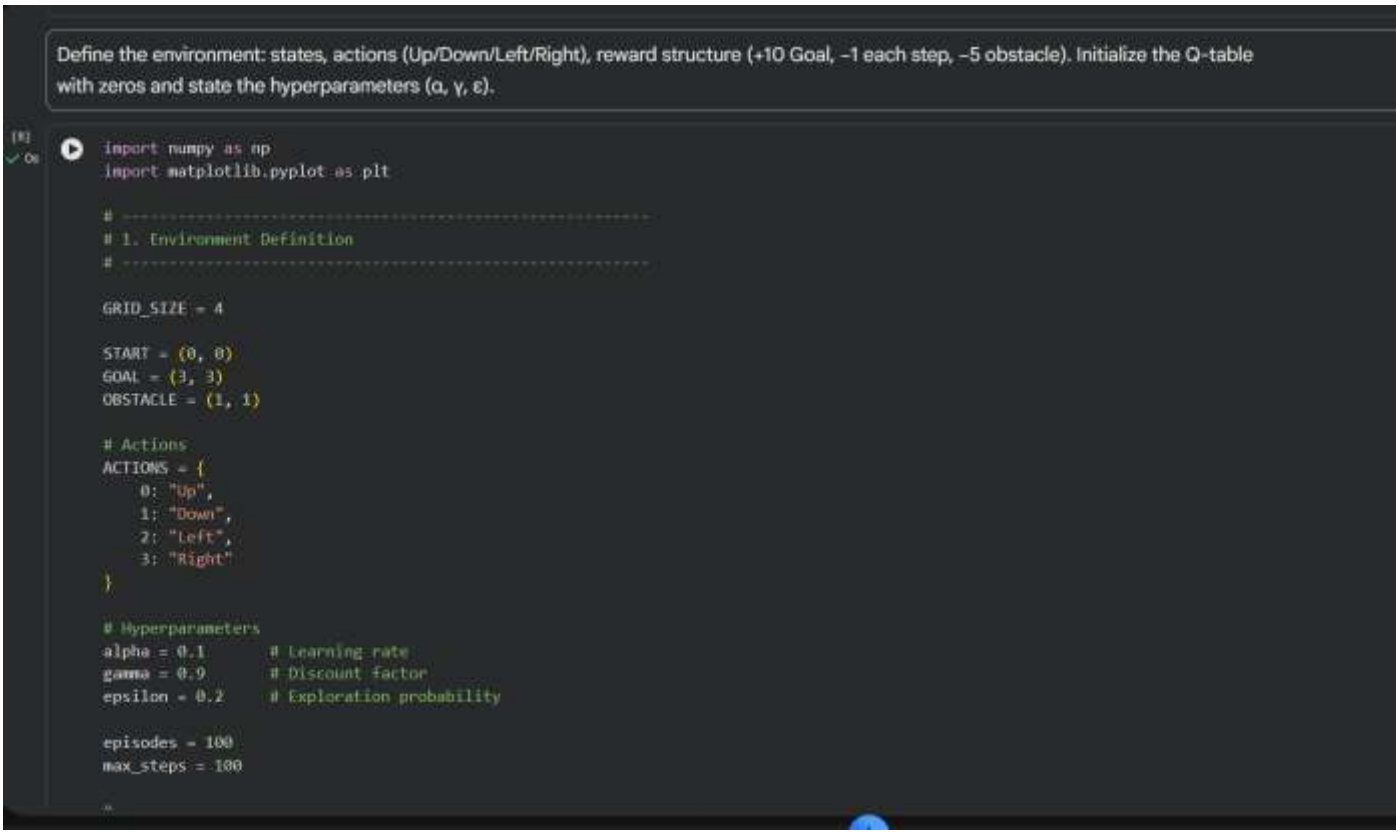

Misclassified Instances:
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
134                6.1               2.6                5.6               1.4
77                 6.7               3.0                5.0               1.7

         Actual   Predicted
134   virginica  versicolor
```

**Experiment 2:** Reinforcement Learning – Q-Learning

***Objective:*** Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).



[8] ✓ 0s

```
# ------------------------------------------------------------
# 2. State Conversion
# ------------------------------------------------------------

def state_to_index(state):
    """Convert (row, col) into state number."""
    return state[0] * GRID_SIZE + state[1]


def index_to_state(index):
    """Convert state number into (row, col)."""
    return divmod(index, GRID_SIZE)


# ------------------------------------------------------------
# 3. Environment Step
# ------------------------------------------------------------

def step(state, action):

    row, col = state

    # Calculate new position
    if action == 0:       # Up
        new_row, new_col = row - 1, col

    elif action == 1:     # Down
        new_row, new_col = row + 1, col

    elif action == 2:     # Left
        new_row, new_col = row, col - 1

    else:                 # Right
        new_row, new_col = row, col + 1
```

[8] ✓ 0s

```
    else:                 # Right
        new_row, new_col = row, col + 1

    # Boundary condition
    if (new_row < 0 or new_row >= GRID_SIZE or
        new_col < 0 or new_col >= GRID_SIZE):

        # Stay in the same state
        return state, -1, False

    new_state = (new_row, new_col)

    # Goal
    if new_state == GOAL:
        return new_state, 10, True

    # Obstacle
    if new_state == OBSTACLE:
        return state, -5, False

    # Normal movement
    return new_state, -1, False


# ------------------------------------------------------------
# 4. Initialize Q-table
# ------------------------------------------------------------

# 16 states x 4 actions
Q = np.zeros((GRID_SIZE * GRID_SIZE, len(ACTIONS)))

print("Initial Q-table:")
print(Q)
```

[8] ✓ 0s

```
# ------------------------------------------------------
# 5. Q-Learning
# ------------------------------------------------------

cumulative_rewards = []

# Store Q-values for representative states
q_history_state_0 = []
q_history_state_5 = []

for episode in range(1, episodes + 1):

    state = START
    total_reward = 0

    for step_number in range(max_steps):

        state_index = state_to_index(state)

        # Epsilon-greedy action selection
        if np.random.random() < epsilon:
            action = np.random.choice(list(ACTIONS.keys()))
        else:
            action = np.argmax(Q[state_index])

        # Take action
        next_state, reward, done = step(state, action)

        next_state_index = state_to_index(next_state)

        # Q-learning update
        if done:
            target = reward
        else:
            target = reward + gamma * np.max(Q[next_state_index])

        Q[state_index, action] = (
            Q[state_index, action]
            + alpha * (target - Q[state_index, action])
        )

        total_reward += reward
        state = next_state

        if done:
            break

    cumulative_rewards.append(total_reward)

    # Record Q-values for representative states
    q_history_state_0.append(Q[0].copy())
    q_history_state_5.append(Q[5].copy())


# ------------------------------------------------------
# 6. Display Q-values at Episodes 1, 50 and 100
# ------------------------------------------------------

print("\nQ-values for State 0:")
print("Episode 1  :", q_history_state_0[0])
print("Episode 50 :", q_history_state_0[49])
print("Episode 100:", q_history_state_0[99])

print("\nQ-values for State 5:")
print("Episode 1  :", q_history_state_5[0])
print("Episode 50 :", q_history_state_5[49])
print("Episode 100:", q_history_state_5[99])
```

[8] ✓ 0s

```
# ------------------------------------------------------------
# 7. Display Final Q-table
# ------------------------------------------------------------

print("\nFinal Q-table:")
print(np.round(Q, 2))


# ------------------------------------------------------------
# 8. Extract Learned Policy
# ------------------------------------------------------------

print("\nLearned Policy:")

policy = []

for state_index in range(16):

    state = index_to_state(state_index)

    if state == GOAL:
        action_name = "Goal"

    elif state == OBSTACLE:
        action_name = "Obstacle"

    else:
        best_action = np.argmax(Q[state_index])
        action_name = ACTIONS[best_action]

    policy.append(action_name)

    print(
        f"State {state_index:2d} {state}: "
        f"{action_name}"
```

[8]
✓ 0s

```
            f"{action_name}"
        )


    # ------------------------------------------------------------
    # 9. Display Policy as Grid
    # ------------------------------------------------------------

    policy_grid = np.array(policy).reshape(4, 4)

    print("\nPolicy Grid:")
    print(policy_grid)


    # ------------------------------------------------------------
    # 10. Plot Cumulative Reward
    # ------------------------------------------------------------

    plt.figure(figsize=(10, 5))

    plt.plot(
        range(1, episodes + 1),
        cumulative_rewards
    )

    plt.xlabel("Episode")
    plt.ylabel("Cumulative Reward")
    plt.title("Q-Learning: Cumulative Reward per Episode")
    plt.grid(True)

    plt.show()
```

```
Initial Q-table:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
```

```
Initial Q-table:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

Q-values for State 0:
Episode 1  : [-0.2881 -0.28   -0.2881 -0.271 ]
Episode 50 : [-2.33968101 -2.17354544 -2.33921837 -2.16074206]
Episode 100: [-2.07077055 -2.18033874 -2.38059645  1.16144196]

Q-values for State 5:
Episode 1  : [0. 0. 0. 0.]
Episode 50 : [0. 0. 0. 0.]
Episode 100: [0. 0. 0. 0.]

Final Q-table:
[[-2.07 -2.18 -2.38  1.16]
 [-0.83 -2.63 -1.37  2.77]
 [ 0.48  4.42 -0.66 -0.89]
 [-0.57  0.46 -0.33 -0.66]
 [-1.68 -1.33 -1.64 -1.94]
 [ 0.    0.    0.    0.  ]
 [ 0.19  6.14 -0.72  0.81]
 [-0.55  5.42 -0.29 -0.2 ]
 [-1.05 -0.82 -0.86  0.37]
```

```
 [ 0.19  0.14 -0.72  0.81]
 [-0.55  5.42 -0.29 -0.2 ]
 [-1.05 -0.82 -0.86  0.37]
 [-0.5  -0.32 -0.33  3.95]
 [ 0.76  2.21  0.23  7.99]
 [ 0.25 10.    1.11  5.54]
 [-0.51 -0.58 -0.63 -0.54]
 [-0.29 -0.37 -0.21  0.72]
 [-0.1   0.81 -0.1   7.46]
 [ 0.    0.    0.    0.  ]]

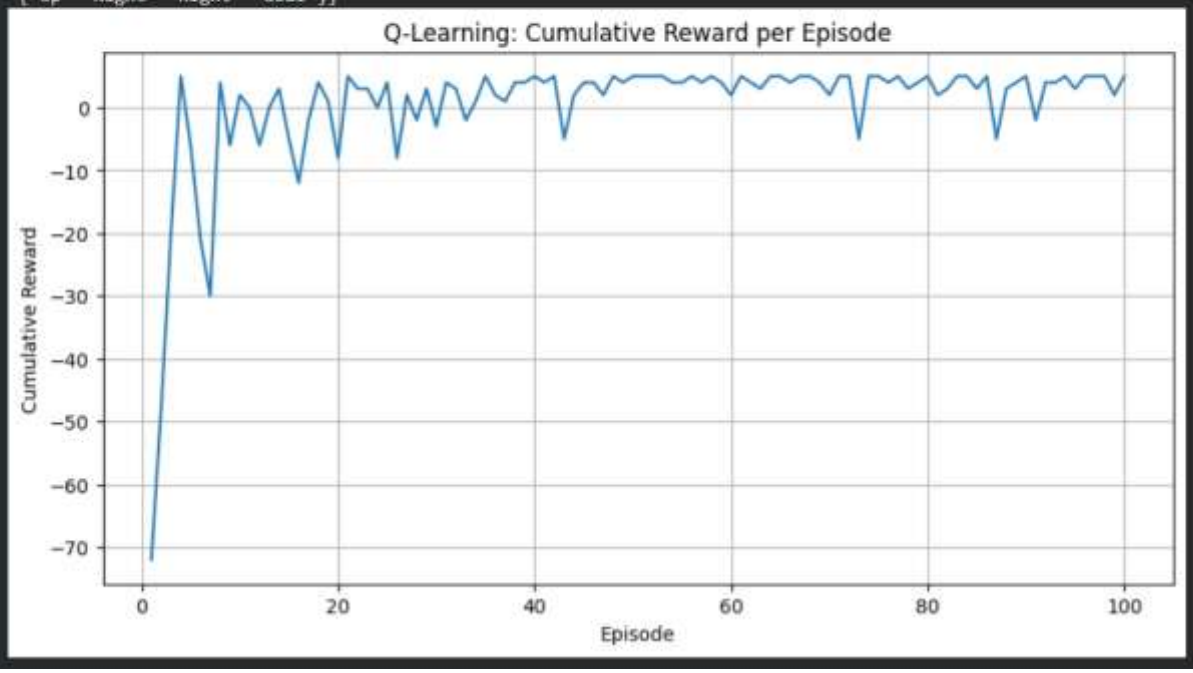
Learned Policy:
State  0 (0, 0): Right
State  1 (0, 1): Right
State  2 (0, 2): Down
State  3 (0, 3): Down
State  4 (1, 0): Down
State  5 (1, 1): Obstacle
State  6 (1, 2): Down
State  7 (1, 3): Down
State  8 (2, 0): Right
State  9 (2, 1): Right
State 10 (2, 2): Right
State 11 (2, 3): Down
State 12 (3, 0): Up
State 13 (3, 1): Right
State 14 (3, 2): Right
State 15 (3, 3): Goal

Policy Grid:
[['Right' 'Right' 'Down' 'Down']
 ['Down' 'Obstacle' 'Down' 'Down']
 ['Right' 'Right' 'Right' 'Down']
 ['Up' 'Right' 'Right' 'Goal']]
```




```
State 14 (3, 2): Right
State 15 (3, 3): Goal

Policy Grid:
[['Right' 'Right' 'Down' 'Down']
 ['Down' 'Obstacle' 'Down' 'Down']
 ['Right' 'Right' 'Right' 'Down']
 ['Up' 'Right' 'Right' 'Goal']]
```




(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

Run Q-Learning for 100 Episodes

```
print("Q-values for State 0:")
print("Episode 1  :", q_history_state_0[0])
print("Episode 50 :", q_history_state_0[49])
print("Episode 100:", q_history_state_0[99])

print("\nQ-values for State 4:")
print("Episode 1  :", q_history_state_5[0])
print("Episode 50 :", q_history_state_5[49])
print("Episode 100:", q_history_state_5[99])
```

```
Q-values for State 0:
Episode 1  : [-0.2881 -0.28   -0.2881 -0.271 ]
Episode 50 : [-2.33968101 -2.17354544 -2.33921837 -2.16074206]
Episode 100: [-2.07077055 -2.18033874 -2.38059645  1.16144196]

Q-values for State 4:
Episode 1  : [0. 0. 0. 0.]
Episode 50 : [0. 0. 0. 0.]
Episode 100: [0. 0. 0. 0.]
```

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

```
plt.figure(figsize=(5, 2))

plt.plot(
    range(1, episodes + 1),
    cumulative_rewards
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid(True)

plt.show()
```



## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Concepts | 25 | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design | 30 | Well-planned, systematic implementation with optimal solution | Correct implementation with minor issues | Basic implementation with errors or inefficiencies | Incorrect or incomplete implementation |
| Use of Tools | 20 | Effective and advanced use of tools/technologies with proper configuration | Appropriate use of tools with correct execution | Limited or inconsistent use of tools | Incorrect or minimal use of tools |
| Analysis of Results | 15 | Insightful analysis with accurate interpretation and validation of results | Good analysis with reasonable interpretation | Basic analysis with limited insights | Poor or incorrect analysis of results |
| Documentation | 10 | Clear, well-structured documentation with diagrams, code, and results | Organized documentation with necessary details | Basic documentation with missing details | Poor or incomplete documentation |